Laboratório Funcional · Aula 12

Eduarda Pereira de Moraes · RA 24170522-2 · Turma ESOFT6S-B
Gerado em 16/10/2026, 20:07:19 · tempo registrado no cronômetro: 46:47

Preparação: Programar com funções matemáticas

def area(r): return 3.14159 * r * r

Pura

def agora(): return datetime.now()

Impura

def total(lista): return sum(lista)

Pura

def registra(msg): print(msg)

Impura

def sorteia(): return random.randint(1, 6)

Impura

def poe(lista, x): lista.append(x); return lista

Impura

0.1 Escolha duas funções que você marcou como impuras e explique, para cada uma, o que a impede de ser pura.

A função agora() é impura porque depende do horário atual, então pode retornar resultados diferentes.
Já a função registra(msg) é impura porque utiliza print causando um efeito colateral ao escrever na tela.

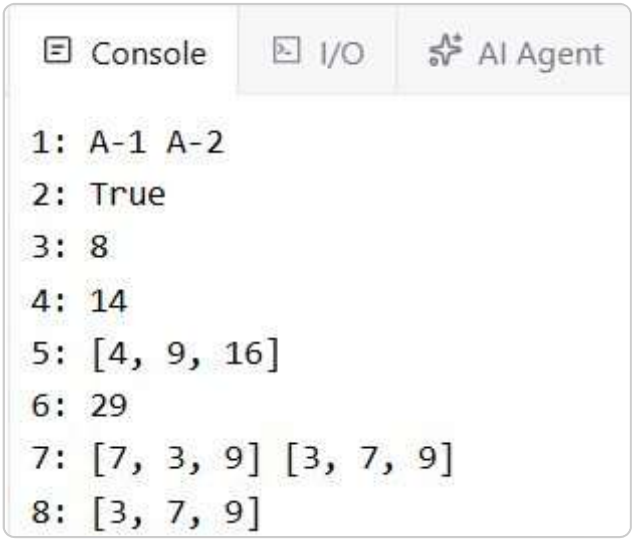
Prática 1: Funções puras e formas funcionais

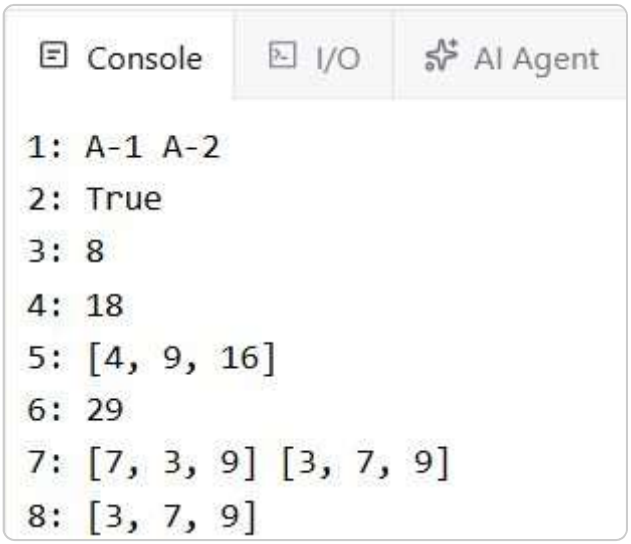
Previsão: O que as linhas 1, 4 e 7 vão imprimir?

Linha 1: A-1 A-2
Linha 4: 14
Linha 7: [7, 3, 9] [3, 7, 9]

Saídas (original e com compor(g, f))

```
Print:
```





1.1 Por que proximo_id("A") devolve valores diferentes para a mesma entrada? Por que isso dificulta os testes (p. 637)?

Porque usa uma variável global que aumenta a cada chamada, que dificulta os testes pois o resultado depende das execuções anteriores.

1.2 compor(f, g)(4) e compor(g, f)(4) deram valores diferentes. Calcule os dois à mão e explique a diferença.

compor(f, g)(4) = f(12) = 14
compor(g, f)(4) = g(6) = 18
Os resultados são diferentes porque a ordem de aplicação das funções muda

1.3 Qual é a diferença entre sorted(notas) e notas.sort()? Qual dos dois segue o estilo funcional, e por quê?

sorted(notas) cria uma nova lista ordenada sem modificar a original seguindo o estilo funcional, já notas.sort() altera diretamente a lista original.

Prática 2: Scheme: tudo é lista

Previsão: (car '(A B C))

A

Previsão: (cdr '(A B C))

(B C)

Previsão: (cons '(A B) '(C D))

((A B) C D)

Saídas (original e sem o apóstrofo)

```
Print:
```

```
Output:
22
14
25
8
negativo
A
(B C)
(A B C)
((A B) C D)
B
```

```
Output:
22
14
25
8
negativo
Backtrace:
           5 (primitive-load "/box/main.scm")
In ice-9/eval.scm:
   191:35  4 (_ #f)
   191:35  3 (_ #f)
   196:27  2 (_ #f)
   223:20  1 (proc #<directory (guile-user) 98722504c80>)
In unknown file:
           0 (%resolve-variable (7 . A) #<directory (guile-user) 987…>)

ERROR: In procedure %resolve-variable:
Unbound variable: A
```

2.1 Para que serve o apóstrofo (')? O que o Scheme tentou fazer com (A B C) quando ele não estava lá?

Com o apóstolo a lista é tratada como dado e não como código. Sem ele o Scheme tentou executar A como uma função causando o erro

2.2 Escreva uma expressão que devolva o terceiro elemento de '(A B C) usando só car e cdr. Teste no OneCompiler e cole aqui a expressão e o resultado.

```
(define (mostra x)
  (display x)
  (newline))

(mostra
  (car
    (cdr
      (cdr '(A B C)))))

Resultado: C
```

Prática 3: Recursão e recursão em cauda

O que vi no passo a passo

n = 4: comum em 8 passos, até 3 multiplicações esperando (expressão de até 30 caracteres); em cauda em 6 passos, nada esperando (até 17 caracteres).
n = 6: comum em 12 passos, até 5 multiplicações esperando (expressão de até 42 caracteres); em cauda em 8 passos, nada esperando (até 18 caracteres).

Previsão: O programa em Scheme chama conta-ate recursivamente um milhão de vezes. Vai funcionar? E a mesma função em Python?

O Scheme vai executar um milhão de chamadas utilizando recursão em cauda. O Python provavelmente apresentará um erro por ultrapassar o limite de recursão.

Saída do Scheme

```
Print:
```

```
Output:
#t
#f
2432902008176640000
2432902008176640000
1000000
```

Saída do Python (as últimas linhas bastam)

```
Print:
```

Console I/O AI Agent

```
500
Traceback (most recent call last):
  File "/box/main.py", line 9, in <module>
    print(conta_ate(1_000_000, 0))
          ^^^^^^^^^^^^^^^^^^^^^^^
  File "/box/main.py", line 6, in conta_ate
    return conta_ate(n - 1, acc + 1)     # a chamada é a última operação
           ^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/box/main.py", line 6, in conta_ate
    return conta_ate(n - 1, acc + 1)     # a chamada é a última operação
           ^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/box/main.py", line 6, in conta_ate
    return conta_ate(n - 1, acc + 1)     # a chamada é a última operação
           ^^^^^^^^^^^^^^^^^^^^^^^^^^
  [Previous line repeated 996 more times]
RecursionError: maximum recursion depth exceeded
```

Debug this error

3.1 No passo a passo, o que cresce na recursão comum e não cresce na recursão em cauda? Por quê?

Na recursão comum a pilha cresce porque existem operações pendentes. Na recursão em cauda a chamada recursiva é a última operação permite que a pilha não cresça quando a linguagem faz essa otimização.

3.2 Por que o Scheme fez um milhão de chamadas e o Python parou com RecursionError? (Dica: releia o quadro sobre a otimização da chamada em cauda e a mensagem de erro do Python. O livro fala do Scheme na p. 656.)

Porque o Scheme otimiza a recursão em cauda evitando o crescimento da pilha, o Python não realiza essa otimização e possui um limite de recursão causando o erro.

Prática 4: Mão na massa: três funções sobre listas

Saída final (com os três ok)

```
Print:
```

Output:

```
tamanho: 3  ok
soma-quadrados: 14  ok
pares: (2 4 6)  ok
```

4.1 Cole aqui as suas três funções (tamanho, soma-quadrados e pares).

```
(define (tamanho lis)
  (if (null? lis)
      0
      (+ 1 (tamanho (cdr lis)))))

(define (soma-quadrados lis)
  (if (null? lis)
      0
      (+ (* (car lis) (car lis))
         (soma-quadrados (cdr lis)))))

(define (pares lis)
  (cond
    ((null? lis) '())
    ((even? (car lis))
     (cons (car lis) (pares (cdr lis))))
    (else (pares (cdr lis)))))
```

4.2 Qual foi a parte mais difícil? Onde você usou null?, car e cdr em cada função?

A parte mais difícil foi entender como processar as listas usando recursão, usei null para verificar se a lista estava vazia, car para acessar o primeiro elemento e cdr para continuar com os elementos restantes. Na função tamanho, usei null e cdr, nas outras, utilizei os três.

Prática 5: Haskell: padrões, compreensões e preguiça

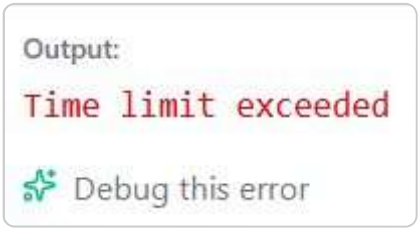
Previsão: squares é uma lista infinita. O que você acha que take 5 squares imprime? E primeiro 42 (div 1 0)?

O take 5 squares imprime [0,1,4,9,16], pois pega apenas os cinco primeiros números da lista de quadrados. Já o primeiro 42 (div 1 0) retorna 42, sem apresentar erro de divisão por zero, porque o Haskell não calcula o segundo parâmetro, já que ele não é utilizado

Saídas (original e com member 17)

```
Print:
```

```
Output:
2432902008176640000
[1,2,4,7,14]
[2,2,4,7,8,9]
[0,1,4,9,16]
[0,1,4,9,16,25,36,49]
True
42
```



5.1 Como o Haskell consegue imprimir parte de uma lista infinita? E por que primeiro 42 (div 1 0) não deu erro de divisão por zero?

O Haskell utiliza avaliação preguiçosa, ou seja, só calcula os valores quando eles são necessários. Por isso, mesmo que a lista seja infinita, o comando take 5 squares consegue mostrar apenas os cinco primeiros elementos. O comando primeiro 42 (div 1 0) não apresenta erro porque a função utiliza apenas o primeiro parâmetro, que é 42. Como o segundo parâmetro é ignorado, a divisão por zero não é executada.

5.2 O que aconteceu com member 17 squares? Por que ele nunca termina, enquanto member 16 squares terminou? Como você corrigiria? (Dica: squares está em ordem crescente. Quando já dá para desistir da busca? O livro mostra uma solução na p. 670.)

O comando member 17 squares fica procurando o número 17 em uma lista infinita, mas não consegue encontrá-lo, pois 17 não é um quadrado perfeito. Por isso, a busca não termina. Já o member 16 squares termina porque o número 16 existe na lista. Para corrigir, podemos aproveitar que a lista está em ordem crescente. Assim, quando o número da lista passar do valor procurado, a busca pode parar e retornar False.

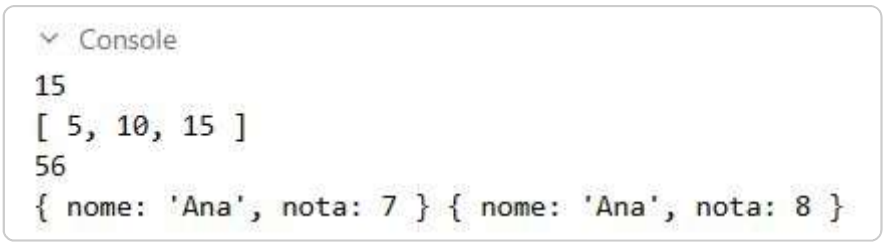
Prática 6: Você já usa funcional

Previsão: O que mais10(5) e [1, 2, 3].map(vezes(5)) vão imprimir?

O mais10(5) vai imprimir 15, porque soma 10 ao número 5. Já o [1, 2, 3].map(vezes(5)) vai imprimir [5, 10, 15], pois multiplica cada número da lista por 5.

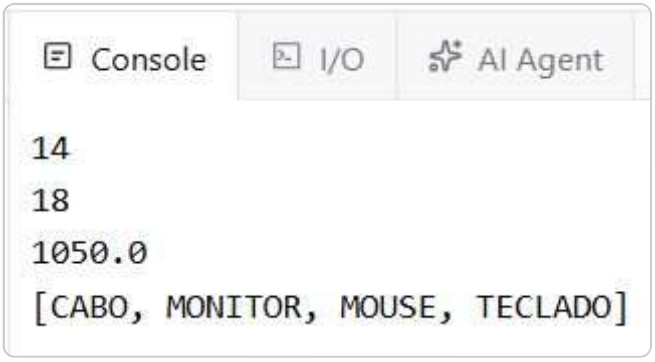
Saída do JavaScript

```
Print:
```



Saída do Java

```
Print:
```



6.1 somador(10) já terminou de executar quando mais10(5) é chamado. Como a função devolvida ainda "lembra" o 10?

Isso acontece por causa do fechamento (closure). Mesmo depois que a função `somador(10)` termina, a função criada continua guardando o valor 10. Assim, quando chamamos `mais10(5)`, ela consegue somar 5 + 10 e retornar 15.

6.2 Por que f.compose(g).apply(4) e f.andThen(g).apply(4) deram resultados diferentes? Qual deles corresponde ao compor(f, g) da Prática 1?

Os resultados são diferentes porque a ordem das funções muda. No f.compose(g), primeiro executa g(4) = 12 e depois f(12) = 14. Já no f.andThen(g), primeiro executa f(4) = 6 e depois g(6) = 18. O f.compose(g) corresponde ao compor(f, g) da Prática 1, considerando que ele aplica primeiro g e depois f.

Prática 7: O laço vira pipeline

7.1 Primeiro exercício escolhido (1, 2 ou 3): cole o seu código reescrito.

```
2 - JavaScript
const alunos = [
  { nome: "Ana", nota: 8.5 },
  { nome: "Bruno", nota: 5.0 },
  { nome: "Carla", nota: 7.0 },
  { nome: "Davi", nota: 9.5 },
];

const aprovados = alunos
  .filter(aluno ⇒ aluno.nota ⩾ 6)
  .map(aluno ⇒ aluno.nome.toUpperCase());

console.log(aprovados.join(", "));
```

7.2 Segundo exercício escolhido: cole o seu código reescrito.

```
3 - python
precos = [50.0, 120.0, 80.0, 200.0]

com_desconto = [p * 0.9 if p > 100 else p for p in precos]

print(com_desconto)

Bloco de escrever execução
JavaScript:
ANA, CARLA, DAVI
```

```
Python:
[50.0, 108.0, 80.0, 180.0]
```

Saídas das suas versões

```
Print:
```





7.3 Qual versão é mais fácil de ler: a com laço ou a funcional? E qual é mais fácil de depurar (por exemplo, para ver um valor intermediário)?

Na minha opinião, a versão funcional é mais fácil de ler porque o código fica menor e mais organizado, sem precisar utilizar laços. Por outro lado, a versão com laço é mais fácil de depurar, pois conseguimos acompanhar cada repetição e verificar os valores intermediários com mais facilidade.

Fechamento: O que fica do paradigma funcional

Uma função pura pode ler uma variável global que muda.

Falso

Em Scheme, a recursão em cauda não faz a pilha crescer.

Verdadeiro

O Python otimiza a recursão em cauda.

Falso

Em Haskell, [1 ..] só é calculada à medida que é usada.

Verdadeiro

filter, map e reduce dispensam laço e variável acumuladora.

Verdadeiro

Currying transforma uma função de vários parâmetros numa cadeia de funções de um parâmetro.

Verdadeiro

8.1 Qual versão é mais fácil de entender, e por quê? O livro estima que um programa funcional costuma levar cerca de 2 vezes mais tempo para executar (p. 678). Em que tipo de sistema você aceitaria esse custo, e em qual não?

Achei a versão em Haskell mais fácil de entender, porque o código é menor e mostra diretamente o que precisa ser feito, sem utilizar laços. Aceitaria esse custo de desempenho em sistemas simples, como aplicativos de cadastro ou sistemas administrativos. Já em sistemas que precisam de respostas muito rápidas, como jogos em tempo real ou sistemas críticos, esse custo poderia ser um problema.

8.2 Em uma frase: o que mais surpreendeu você nesta atividade?

Me surpreendeu foi descobrir que o Haskell consegue trabalhar com listas infinitas sem precisar calcular todos os elementos.

8.3 Qual é o erro de impressão, e como corrigir? O que a mensagem do compilador diz sobre o tipo que ele inferiu?

O erro está no uso de <- no lugar de <=. O correto é usar b <= h para comparar os elementos com o pivô. O compilador interpreta <- como uma operação que retira elementos de uma lista, mas h é um único elemento. Por isso, ocorre um erro de tipos, pois ele esperava uma lista e encontrou um valor individual.